CAPS.ai Meetings Organizer

Setup Guide - macOS and Windows

Updated 29 October 2026. Run the application locally from VS Code with a local PostgreSQL database. The deployed Render database is separate.

Required software

- VS Code and a copy of the CAPS_ai repository.

- .NET 10 SDK (the SDK includes the runtime).

- Node.js 24 with npm, matching this project's Docker build.

- PostgreSQL 16, including command-line tools. Windows users can also install pgAdmin.

- Git if cloning the repository; access is required for the private repository.

Official downloads: .NET 10 SDK, Node.js, PostgreSQL for Windows, Homebrew, VS Code.

For Apple Silicon Macs choose Arm64 software; Intel Macs normally use x64. On Windows choose the installer matching your processor. Restart VS Code after installing software so new PATH settings are available.

Check the installation

Run in the VS Code terminal on either operating system:

```
dotnet --version
node --version
npm --version
```

Windows instructions below use PowerShell. If PowerShell blocks npm.ps1, use npm.cmd in place of npm; changing the execution policy is not necessary.

Project names

```
CAPS_ai/
  CAPS_ai.slnx
  CAPS_ai.API/
  CAPS_ai-frontend/
```

The local database is still named SmartMeetDB. Database configuration uses the key ConnectionStrings:SmartMeetDB. Those names did not change when the project was renamed.

Choose the correct database setup

- Existing Mac installation: start PostgreSQL and the app. Do not recreate the database or run migrations during normal startup. Its migration history was previously removed.

- Fresh installation: create an empty SmartMeetDB, configure the connection, and run the guarded migration command once.

- Existing database with records: do not treat it as empty. If migration history is missing, restore verified matching history before future schema updates.

The following commands do not delete records. Do not run database-reset SQL to fix startup problems.

macOS - setup and startup

1. Open the project

In VS Code use File > Open Folder and select CAPS_ai. For this Mac, the path is:

```
cd /Users/abhilekhborah/Desktop/Assigment_1/CAPS_ai
```

On another Mac, substitute the folder where you downloaded the project. Open separate terminals for the backend and frontend.

2. Start PostgreSQL

If PostgreSQL 16 is not installed and Homebrew is available, run brew install postgresql@16 once. Then:

```
brew services start postgresql@16
export PATH="$(brew --prefix postgresql@16)/bin:$PATH"
pg_isready -h localhost -p 5432
```

The export applies to this terminal. Repeat it in a new terminal if PostgreSQL commands are not found. pg_isready should report accepting connections.

3. Fresh database only

Skip this section on the existing Mac installation. For a new Homebrew setup, the default database role normally matches your macOS username:

```
createdb -h localhost -p 5432 SmartMeetDB
```

If your PostgreSQL server uses another role, add -U YOUR_DB_USER and configure the backend using the connection command below. Replace both placeholders; do not commit a real password:

```
local_db='Host=localhost;Port=5432;Database=SmartMeetDB;'
local_db+='Username=YOUR_DB_USER;Password=YOUR_PASSWORD'
dotnet user-secrets set "ConnectionStrings:SmartMeetDB" "$local_db" --project CAPS_ai.API
unset local_db
```

The default app configuration can be used when your local role and authentication already work without a password. No user-secrets init command is needed: the project already has a UserSecretsId.

4. Start the backend - terminal 1

From the CAPS_ai root:

```
dotnet restore CAPS_ai.slnx
# Fresh empty database only; skip for the existing Mac database:
# dotnet run --project CAPS_ai.API -- --migrate
dotnet run --project CAPS_ai.API
```

For a fresh database, run the commented migration command without the leading # before starting the API. It exits after creating the schema. Keep the normal API process running.

5. Start the frontend - terminal 2

```
cd /Users/abhilekhborah/Desktop/Assigment_1/CAPS_ai/CAPS_ai-frontend
npm ci
npm start
```

Use your own folder path on another Mac. On subsequent starts, npm start is enough unless dependencies changed.

Windows - setup and startup

1. Install PostgreSQL and open the project

Use the installer linked from PostgreSQL's official Windows download page. Select PostgreSQL 16, install the server and command-line tools, keep port 5432 unless already used, and remember the password set for the postgres role. pgAdmin is optional.

Open the CAPS_ai folder in VS Code. The example below assumes C:\Projects\CAPS_ai; replace it with your actual folder. Commands use PowerShell.

```
Set-Location "C:\Projects\CAPS_ai"
$env:Path = "C:\Program Files\PostgreSQL\16\bin;" + $env:Path
pg_isready -h localhost -p 5432
```

2. Start the database service if needed

The installer normally creates a Windows service. If pg_isready reports no response, find its exact name:

```
Get-Service *postgres*
```

In an Administrator PowerShell, run Start-Service with the name shown, for example Start-Service -Name "postgresql-x64-16". Service names can differ. Recheck readiness in VS Code.

3. Create a fresh database and configure the backend

Connect using the password chosen during installation:

```
psql -h localhost -p 5432 -U postgres -d postgres
```

At the psql prompt, create the empty database and exit:

```
CREATE DATABASE "SmartMeetDB";
\q
```

Back in PowerShell at the project root, replace YOUR_PASSWORD with your local password:

```
$localDb = "Host=localhost;Port=5432;Database=SmartMeetDB;"
$localDb += "Username=postgres;Password=YOUR_PASSWORD"
dotnet user-secrets set "ConnectionStrings:SmartMeetDB" $localDb --project CAPS_ai.API
Remove-Variable localDb
```

This stores development configuration outside the repository. User secrets are local files, not an encrypted password vault. Use only local development credentials. If a password contains shell or connection-string special characters, quote and escape it appropriately; never paste the real command into shared reports.

4. Migrate once and start the backend - terminal 1

```
dotnet restore CAPS_ai.slnx
dotnet run --project CAPS_ai.API -- --migrate
dotnet run --project CAPS_ai.API
```

The migration command is for the fresh database above. It safely refuses an existing application schema with missing migration history. Do not run dotnet ef migrations add: the migrations already exist in the project.

5. Start the frontend - terminal 2

```
Set-Location "C:\Projects\CAPS_ai\CAPS_ai-frontend"
npm.cmd ci
npm.cmd start
```

Keep both app terminals running. Windows uses the same backend and frontend ports as macOS.

Inspect tables and data - both platforms

1. Connect to the same local database as the backend

macOS (default local role):

```
psql -h localhost -p 5432 -d SmartMeetDB
```

Windows (or a server configured with the postgres role):

```
psql -h localhost -p 5432 -U postgres -d SmartMeetDB
```

If you configured another database role, use that role with -U. Add PostgreSQL's bin folder to PATH as shown on the corresponding setup page if psql is not found.

2. List tables and columns

Run these commands inside psql, not directly in PowerShell or zsh:

```
\conninfo
\dt
\d "Meeting"
\d "MeetingAttendees"
\d "MeetingQuestions"
```

3. View records

```
SELECT "MeetingId", "Title", "MeetingCode", "Status"
FROM "Meeting" ORDER BY "MeetingId";

SELECT "UserId", "MeetingId", "Name", "Organization", "Role"
FROM "MeetingAttendees" ORDER BY "UserId";

SELECT "QuestionId", "MeetingId", "UserId",
       "QuestionText", "Status"
FROM "MeetingQuestions" ORDER BY "SubmittedAt", "QuestionId";
```

SQL statements need a semicolon. Backslash commands do not. PostgreSQL requires double quotes for these mixed-case identifiers. These example queries omit private token hashes.

```
\x
\pset pager off
\q
```

Use \x for expanded rows, \pset pager off to stop paging, and \q to exit. Disconnecting psql does not stop the database server.

Optional: use pgAdmin

Register a server with host localhost, port 5432, your configured database role, and its password. Expand Databases > SmartMeetDB > Schemas > public > Tables. Right-click a table and use View/Edit Data, or open Query Tool to run the SELECT statements above. Backslash commands such as \dt work only in psql.

Fresh installations have __EFMigrationsHistory as well as the three application tables. Keep that history table. The existing Mac copy may lack it because it was explicitly removed earlier.

Verify, stop, and troubleshoot

Application addresses - both platforms

- Frontend: http://localhost:4200/

- API: http://localhost:5050/api/meetings

- Swagger: http://localhost:5050/swagger/index.html

- Application health: http://localhost:5050/health

- Database readiness: http://localhost:5050/health/ready

Swagger is served by the backend on 5050, not Angular on 4200. The local launch profile selects Development; Swagger is disabled in the public Production deployment.

Check the complete workflow

1. Open the frontend and create a meeting.

2. Keep the original browser session to retain organizer access.

3. Open the invitation in a separate browser profile or private window, then join as a visitor.

4. Submit a question as the visitor. In the organizer view, select the next question and mark it answered.

5. Rerun the SQL queries to see meetings, attendees, and questions in PostgreSQL.

Common problems

- dotnet not found: install the .NET 10 SDK and restart VS Code.

- npm blocked in PowerShell: use npm.cmd ci and npm.cmd start.

- PostgreSQL unavailable: start its service, check pg_isready, and confirm port 5432.

- Password/role errors: configure the local connection with dotnet user-secrets. Match the role used by psql.

- Wrong database selected: DATABASE_URL takes priority over the local connection string. Remove a stale cloud DATABASE_URL from the terminal before running locally. macOS: unset DATABASE_URL. PowerShell: Remove-Item Env:DATABASE_URL -ErrorAction SilentlyContinue.

- Missing tables on a fresh database: run the guarded --migrate command from its setup section.

- Migration refuses an existing schema: restore verified matching migration history before schema updates. Do not replay initial migrations or drop tables as a workaround.

- Frontend API errors: leave the backend running on port 5050; Angular proxies /api requests there.

- Voice input fails in Brave: try Chrome with microphone permission. Browser speech availability depends on the browser and its speech service.

Stop and check ports

Press Ctrl+C in each application terminal. This does not delete database records.

macOS:

```
lsof -nP -iTCP:4200 -iTCP:5050 -sTCP:LISTEN
```

Windows PowerShell:

```
Get-NetTCPConnection -State Listen -LocalPort 4200,5050 -ErrorAction SilentlyContinue
```

Stop an old app terminal before starting a second instance. These commands inspect ports without killing unrelated processes.

Local app versus deployed app

The public site is https://caps-ai-meetings-organizer.onrender.com/. It can be used without running local servers. Its cloud database is separate; local psql queries do not show cloud data. This guide does not change the deployment.

Sources and verification

Commands were checked against the renamed repository, its .NET launch profile, and frontend proxy configuration. The macOS Docker deployment was previously verified; Windows commands are documented for PowerShell but were not executed on a Windows machine in this session.

Sources: PostgreSQL Windows installer, .NET SDK, Node.js downloads, ASP.NET Core development secrets.